



Die Jeff-Erweiterung ist eine eigenständige, globale Erweiterung für den pi-Programmier-Agenten. Sie transformiert das Standard-Agentenerlebnis in einen hochgradig strukturierten, selbstorganisierenden Entwickler-Assistenten, indem sie ein "Gehirn" (Arbeitsspeicher) und "Wissenselemente" (Langzeitgedächtnis) einführt.

Warum existiert diese Erweiterung?

Standardmäßig beginnt ein KI-Programmier-Assistent jede Sitzung von Grund auf neu und verlässt sich vollständig auf den unmittelbaren Kontext, den Sie bereitstellen. Bei komplexen Aufgaben, die mehrere Sitzungen umfassen oder architektonische Tiefe erfordern, kann dies zu Folgendem führen:

Verlorener Kontext: Der Agent vergisst Entwurfsmuster oder Entscheidungen aus früheren Sitzungen. Unstrukturierte Ausführung: Der Agent beginnt möglicherweise sofort mit der Änderung des Codes, ohne die Architektur vollständig zu verstehen oder Ihre Zustimmung einzuholen. Verstreute Artefakte: Wichtige Dokumente wie Aufgabenlisten und Entwurfspläne werden zwar erstellt, aber nicht dauerhaft nachverfolgt. Jeff löst dies durch die Einführung von:

Planungsmodus: Zwingt das LLM dazu, bei komplexen Aufgaben innenzuhalten und einen architektonischen Plan zu formulieren (mit Benutzerfreigabe), bevor der Codebase geändert wird. **Persistentes Wissen (KIs):** Ermöglicht es dem Agenten, "Wissenselemente" (metadata.json) sitzungsbereitend zu lesen und zu schreiben, sodass er im

Laufe der Zeit die spezifischen Muster Ihres Repositorys lernt.**Strukturiertes Gehirn:** Leitet Tracking-Dokumente (task.md, implementation_plan.md, walkthrough.md) automatisch in ein dediziertes Verzeichnis ~/.jeff/brain/<session-id>/ um, damit Ihr Projektstammverzeichnis sauber bleibt, während der Agent organisiert ist.**Technische Details**Die Erweiterung ist ein TypeScript-Modul, das beim Start dynamisch von pi geladen wird. Sie befindet sich in Ihrem globalen Erweiterungsverzeichnis unter ~/.pi/agent/extensions/jeff.ts.

1. Kontext-Injektion (beforeagentstart)

Die Erweiterung hängt sich in das Ereignis beforeagentstart ein. Bevor das LLM ein einziges Token generiert, modifiziert Jeff den System-Prompt, um Folgendes einzufügen:

Arbeitsbereich-Metadaten: Fügt das aktuelle Betriebssystem (os.platform()), die lokale Zeit und den aktiven Benutzernamen ein, um dem Modell einen realen Kontext zu geben.**KI-Zusammenfassungen:** Scannt ~/.jeff/knowledge/ nach metadata.json-Dateien. Wenn frühere Erkenntnisse gefunden werden, werden einzeilige Zusammenfassungen dieser Konzepte in den Prompt eingefügt.**Artefakt-Definitionen:** Teilt dem Agenten genau mit, wo sich sein Arbeitsspeicher (~/.jeff/brain/<session-id>/) für die aktuelle Sitzung befindet.**Verhaltensrichtlinien:** Hängt einen strengen <planningmode>-Regelsatz und ein <slashcommands>-Menü an den System-Prompt an.

2. Slash-Befehle

Die Erweiterung registriert benutzterdefinierte interaktive Befehle über `pi.registrierCommand` und verwendet native Befehle, um den Agenten zu steuern:

- `/plan [Aufgabe] ->` Veranlasst den Agenten, einen `implementation_plan.md` im Gehirn-Verzeichnis für die angegebene Aufgabe zu erstellen.

- `/distill ->` Weist den Agenten an, die wichtigsten Erkenntnisse der aktuellen Sitzung zu extrahieren und in ein neues Wissensselement zu schreiben.
- `/grill-me ->` Fordert den Agenten auf, in einen interaktiven Interview-Modus zu wechseln, um unpräzise Anforderungen zu klären, bevor Code geschrieben wird.

- `/goal [Ziel] ->` Weist den Agenten an, autonom auf ein langfristiges Ziel hinzuarbeiten, ohne anzuhalten, bis es vollständig erreicht ist.
- `/schedule [Zeit] [Prompt] ->` Legt einen wiederkehrenden Zeitplan oder einen einmaligen Timer fest, damit der Agent eine bestimmte Anweisung im Hintergrund ausführt.

3. Verzeichnisstruktur

Die Erweiterung verwaltet automatisch die folgenden Verzeichnisse:

`~/jeff/brain/<session-id>/` - Ephemerer Arbeitsspeicher pro Sitzung (Aufgabenlisten, Pläne, Anleitungen).`~/jeff/knowledge/` - Langfristiger, persistenter Speicher über

Werkzeugset von `pi`, um Ihr Repository zu untersuchen, herauszufinden, welchen Stack Sie verwenden, und seine Arbeit unabhängig zu verifizieren. Während der "Verify"-Phase des Workflows wird der Agent: Das `bash-Werkzeug` nutzen, um die native Testsuite Ihres Projekts auszuführen (egal ob `npm run test`, `pytest`, `cargo test`, `go test`, etc.).Kompilierungsprüfungen oder für Ihre Sprache spezifische Typ-Checker ausführen. Wenn ein Fehler auftritt, wird der Output direkt an das Kontextfenster des Agenten zurückgegeben. Er liest den Stack-Trace, analysiert das Versagen und verwendet seine Bearbeitungswerkzeuge, um den Bug autonom zu iterieren und zu beheben. Wie kann ich verhindern, dass der Agent in einer Schleife stecken bleibt?Gelegentlich kann ein LLM in einer Schleife gefangen sein (z. B. wiederholtes Anwenden desselben Fixes, der einen Testfehler nicht behebt). So verwalten Sie dies: **Unterbrechen und Steuern:** Sie haben immer die Kontrolle. Wenn Sie sehen, dass der Agent schleift, drücken Sie `Strg+C` in Ihrem Terminal, um die Tool-Ausführung zu unterbrechen. Sie können dann einen Hinweis geben wie "Du drehst dich im Kreis. Das Problem liegt eigentlich im Datenbankschema, nicht im Route-Handler."Kontext-Kompression: `pi` komprimiert den Kontext nativ. Wenn eine Schleife zu lange dauert, wird alter Kontext gelöscht, was die Schleife manchmal organisch beenden kann.Interaktive Befragung: Wenn Sie eine komplexe Debugging-Sitzung erwarten, verwenden Sie vorher `/grill-me`, um sicherzustellen, dass der Agent die Edge-Cases vollständig versteht, bevor er Code schreibt, was die Wahrscheinlichkeit von zyklischem Debugging erheblich verringert.

Syntaxprüfungen nicht bestanden. Sie MÜSSEN diesen Fehler beheben, bevor Sie fortfahren.

Jeff (Agent):

Liest den injizierten Fehler-Stack-Trace, erkennt den Fehler und sendet einen zweiten Werkzeug-Aufruf, um den Bug zu beheben, bevor der Turn abgeschlossen wird.

Sie (Benutzer):

Sehen nur den finalen, perfekt funktionierenden Code, ohne zu wissen, dass der Agent gezwungen wurde, seine eigenen Fehler im Hintergrund zu korrigieren!

FAQ: Den Agenten verwalten

Wie genehmige oder ändere ich einen Plan, damit der Agent mit der Entwicklung beginnt? Da die injizierten

`<planningmode>`-Regeln den Agenten anweisen, vor der Ausführung *“Benutzerfreigabe einzuholen”*, stoppt der Agent nach dem Schreiben des `implementationplan.md` und wartet. Um ihn zu genehmigen, geben Sie einfach eine Antwort im Terminal wie:

“Sieht gut aus, mach einfach weiter.” Wenn Sie den Plan vor der Ausführung ändern möchten: Sagen Sie dem Agenten einfach, was sich ändern soll! Da der Agent pausiert und auf Ihr Feedback wartet, können Sie antworten mit:

“Bevor wir ausführen, ändere den Plan so, dass `localStorage` statt einer Datenbank verwendet wird.” “Schritt 2 gefällt mir nicht, lass uns diesen Teil so umschreiben, dass eine CSS-Animation verwendet wird.” Der Agent liest Ihr Feedback, aktualisiert das Artefakt `implementation_plan.md` mit dem neuen Ansatz und pausiert erneut für Ihre endgültige Genehmigung. Sobald Sie zufrieden sind, sagen Sie “Approved” und er geht in die “Execute”-Phase über.

Wie debuggt der Agent, was er entwickelt hat? Der Agent ist völlig sprach- und framework-agnostisch. Er nutzt das native

Sitzungen hinweg (Entwurfsentscheidungen, Fehlerbehebungen, Regeln für den Codebase). **4. Harness-Orchestrierung (Selbstkorrektur, JIT-Erinnerungen & BFS-Erkundung)** Jeff fungiert als fortschrittlicher Orchestrator, der Qualität und Kontext während der Ausführung strikt durchsetzt:

Die Verifizierschleife (Dual-Layer Phase Gate): Wann immer der Agent Code modifiziert, fängt Jeff den Werkzeug-Output ab. Zuerst sucht es nach einem `.pify.sh`-Skript, um benutzerdefinierte Projektprüfungen auszuführen. Wenn keines gefunden wird, greift es auf Zero-Config-Sprach-Linter (wie `ruff` für Python oder `tidy` für HTML) zurück. Wenn ein Check fehlschlägt, werden die Protokolle direkt als harter Fehler an den Agenten zurückgegeben, was ihn zwingt, sich selbst zu korrigieren und die Fehler zu beheben, bevor er fortfahren darf. **Just-In-Time (JIT) Kontext-Erinnerungen:** Anstatt den System-Prompt mit riesigen Regelbüchern zu überladen, injiziert Jeff dynamisch gezielte Mikro-Erinnerungen in das Aufmerksamkeitsfenster des Agenten, genau dann, wenn sie benötigt werden. Zum Beispiel erhält der Agent beim Bearbeiten einer `.ts`-Datei die sofortige Anweisung, Inline-Importe und einzeilige Hilfsfunktionen zu vermeiden. **Aufgabenspezifisches Modell-Routing (SLM-Delegation):** Jeff wechselt das aktive LLM dynamisch basierend auf dem Aufgabentyp. Komplexe Aufgaben (Codierung, Debugging) verwenden Ihr schweres primäres Modell, während einfachere Aufgaben (wie die Slash-Befehle `/plan` oder `/distill`) automatisch an schnellere, kostengünstigere Small Language Models (SLMs) weitergeleitet werden, die in `jeffmodelrouting.md` konfiguriert sind. **Code-Graph BFS-Erkundung:** Jeff ändert grundlegend, wie der Agent den Codebase erkundet, indem er den “Codebase Archaeologist”-Constraint injiziert. Dem Agenten ist es streng untersagt, ganze Dateien über 50 Zeilen linear zu lesen. Stattdessen muss er Abhängigkeiten durch zeilenweises Slicing nachverfolgen und seine Hypothese

1. Initiierung einer komplexen Aufgabe

intern dokumentieren, bevor er Lesezugriffe ausführt.get_outline Tool: Jeff registriert ein benutzerdefiniertes Werkzeug, das als struktureller Parser fungiert. Es ermöglicht dem Agenten, das Skelett einer Datei (Klassen, Funktionen, Schnittstellen) zu extrahieren, ohne sein Kontextfenster mit dem Quellcode zu belasten, was chirurgisch präzise Lesezugriffe ermöglicht.BeispielverwendungSobald die Erweiterung in ~/pi/agent/extensions/jeff.ts installiert ist, wird sie bei jedem Start von pi automatisch ausgeführt.

Wenn Sie pi bitten, ein wichtiges Feature zu implementieren, verhindern die injizierten Planungsmodus-Regeln, dass sofort Code geschrieben wird. Stattdessen wird ein Plan ausgegeben.
Sie:
"Refaktoriisiere den Authentifizierungsfluss, um JWT statt Sessions zu verwenden."
Jeff (Agent):

"Da dies eine große architektonische Änderung ist, wechsle ich in den Planungsmodus. Ich werde zuerst den Codebase recherchieren und einen implementation_plan.md in ~/jeff/brain/<session-id>/ zur Überprüfung durch Sie erstellen."

2. Verwendung von Slash-Befehlen

3. Nächste Sitzung

Sie können den Agenten manuell über die registrierten Slash-Befehle im interaktiven Terminal steuern.
Sie:
/plan füge der API eine neue Redis-Caching-Schicht hinzu
Jeff (Agent):
Erstellt einen implementation_plan.md in Ihrem Gehirn-Verzeichnis und bittet um Zustimmung.
Sie:
/distill
Jeff (Agent):
Analysiert die Konversation und schreibt eine neue metadata.json-Datei in ~/jeff/knowledge/redis-caching-pattern/, die zusammenfasst, wie Caching in diesem speziellen Projekt gehandhabt werden sollte.

Wenn Sie pi das nächste Mal starten, liest Jeffs beforeagentstart-Hook das redis-caching-pattern-KI und injiziert dessen Zusammenfassung in den System-Prompt. Der Agent versteht nun nativ Ihre Caching-Regeln, ohne dass Sie sie wiederholen müssen.
4. Die Verifizierschleife (Selbstkorrektur) erlebenFalls der Agent beim Schreiben von Code einen Fehler macht, merken Sie es vielleicht gar nicht. Das Harness fängt es vorher ab: Jeff (Agent):
Verwendet das Bearbeitungswerkzeug, um eine Python-Datei zu ändern, führt aber versehentlich einen Syntaxfehler ein.

Jeff-Erweiterung (Harness):
Fängt die Änderung ab, führt im Hintergrund lautlos den Sprach-Linter aus. Der Check schlägt fehl. Injiziert einen Fehler direkt zurück in den Kontext des Agenten: [Harness Verifier Phase Gate Failed] Ihre Python-Code-Änderung hat